# Software Project

Hamza Jamil, Pesara Amarasekera

March 2026

## 1 Introduction

Isabelle/HOL is a proof assistant and formal system that is a variant of HOL. We can split the discussion of Isabelle/HOL as follows. **Isabelle** is an interactive theorem prover written in Standard ML. It was developed by researchers at the University of Cambridge (Larry Paulson), TU München (Tobias Nipkow), and Université Paris-Sud (Makarius Wenzel).

It is a generic system used as a specification and verification system and for implementing logical formalism.

Isabelle uses Isar as its proof language, which structures proofs to be human-readable.

Isabelle is used widely both in industry, for instance in verifying operating system kernels [2], and in academia, such as in the formal verification of the Kepler Conjecture [1].

**HOL** stands for Higher-Order Logic, and is used for expressing theorems in Isabelle.

$$\text{HOL} = \text{Functional Programming} + \text{Logic}$$

HOL provides a unified language for definitions and proofs. This allows programs, functions and formulas to live within the same framework. HOL is a typed logic and implements STT. Due to HOL's structure we can treat functions and formulas (terms of type bool) as first-class objects.

Isabelle's implementation in ML influences some of HOL's concrete syntax.

For a video introduction to Isabelle/HOL, see [4].

## 2 Theory and Definition Development

In Isabelle as well as in STT, every expression must be well-typed. Types prevent the introduction of meaningless expressions, and provide a guarantee at the typechecker level. Isabelle also supports type inference, allowing the user to simplify their codes and proofs, as well as construct them.

Like most functional programming languages Isabelle/HOL uses type variables for polymorphism. HOL is a logic of total functions; this implies that recursive definitions in HOL must always terminate. There cannot be any inconsistent definitions, and this distinguishes HOL from other systems with STT that allow undefined terms directly.

The translation from Alonzo to HOL is fairly straightforward; we can build our definitions in a similar manner.

We can build our definitions and theorems in an interactive manner: we begin with goals and break them down into subgoals, and keep applying methods, definitions and tactics to resolve these subgoals until no subgoals remain.

$$\text{Goal} \rightarrow \text{Subgoals} \rightarrow \text{No subgoals}$$

[3]

The power of Isabelle/HOL is being able to mechanically verify our theorems using proofs written inside itself. Most of our proofs were able to be handled directly via the `simp` tactic (it transforms our provided assumptions and theorems into the statement we're looking to prove). The ability to do proof searches via tactics like `sledgehammer` was a fun surprise; it's a good demonstration of the brute-force approach to proof search.

We also have the use of other tactics (subprograms for rewriting statements), such as `auto`, which provides more complex automation than `simp` with its ability to handle multiple subgoals at once, and `induct_tac` which handles induction. It is usually the case that we apply a tactic like induction followed by simplification.

Furthermore, the concept of Locales was quite intuitive going from the Alonzo little theories approach to development. I could develop some of the necessary theories and definitions and structures for groups, starting off with a definition of a magma. Without needing to define things twice, much like how parent classes work in OOP and typeclasses work in FP, but for proofs! (Exciting). I followed the following development structure with this regard.

$$magma \rightarrow semigroup \rightarrow monoid \rightarrow group$$

This is akin to how these structures are developed naturally, in order of increasing complexity.

The proof structure of an Isabelle/HOL proof is as follows:

```
lemma app_assoc [simp]: "(xs @ ys) @ zs = xs @ (ys @ zs)"
apply(induct_tac xs)
apply(auto)
done
```

We use the `lemma` keyword or `theorem` for anything we want to prove then we apply tactics until we finish our proof at `done`.

Isabelle organizes theories into files, and theories contain

- types
- functions / definitions
- lemmas and theorems
- proofs

We can see the theory structure below:

```
theory T
  imports Main
begin

(* declarations, definitions, proofs *)

end
```

This type of structure allows our constructs to be modular, while allowing an incremental development cycle, and keeps our artefacts reusable.

Isabelle/HOL allows new concepts to be introduced via definitions, and avoids arbitrary axioms wherever possible. We can develop recursive definitions safely via commands like `primrec`. This kind of development helps preserve consistency across the artefacts produced.

The ability of theories to build on earlier ones is an example of inherited structure in software development; we can also see this structure in Locales.

# 3 Opinion on Isabelle/Jedit interface

The standard interface is called Isabelle/JEdit. I did not enjoy using it that much; it's quite different from using Lean and Agda, or even writing in Haskell. I found inputting unicode to be rather cumbersome, for instance it doesn't simplify input immediately after tabbing, and I had to undo and wait for a dropdown menu to appear to enter the wanted input. Otherwise it's just preference. I found that the cursor-based development (i.e. using the cursor to alter and change proofs) to be rather useful, it's similar to using holes and refinements in Agda.

In the end I think JEdit is a much worse interface because I'm used to the Agda and Lean way of doing things. I never would've thought I would appreciate the work of UI/UX designers (as I'm more of a mathematician), in the course of this project. But UI/UX is an intermediary language and careful work should go into developing these, depending on the audience that you'd be marketing these tools towards. Having a great logic, with fancy functionality is great and all, but if you can't get users to actually use what you have, well that's arguably more concerning.

I think they should use something other than the JVM in developing an Interface for Isabelle, because it's a bit slower than I like it to be.

# 4 Source Code

We present a construction of queues using the internal list

```
theory Scratch
  imports Main
begin

type_synonym 'a queue = "'a list"

definition empty :: "'a queue" where
  "empty = []"

definition enqueue :: "'a queue ⇒ 'a ⇒ 'a queue" where
  "enqueue q x = q @ [x]"

fun dequeue :: "'a queue ⇒ ('a queue × 'a)" where
  "dequeue [] = undefined"
| "dequeue (x # xs) = (xs, x)"

definition front :: "'a queue ⇒ 'a" where
  "front q = snd (dequeue q)"

definition rest :: "'a queue ⇒ 'a queue" where
  "rest q = fst (dequeue q)"

definition singleton :: "'a ⇒ 'a queue" where
  "singleton x = enqueue empty x"

lemma dequeue_enqueue_empty:
  "dequeue (enqueue empty x) = (empty, x)"
  unfolding enqueue_def empty_def
  by simp

lemma front_singleton:
  "front (singleton x) = x"
  unfolding front_def singleton_def
  by (simp add: dequeue_enqueue_empty)

lemma rest_singleton:
  "rest (singleton x) = empty"
  unfolding rest_def singleton_def
  by (simp add: dequeue_enqueue_empty)

lemma front_enqueue_nonempty:
  "q ≠ empty ⟹ front (enqueue q x) = front q"
  unfolding front_def enqueue_def empty_def
  by (cases q) simp_all

lemma rest_enqueue_nonempty:
  "q ≠ empty ⟹ rest (enqueue q x) = enqueue (rest q) x"
  unfolding rest_def front_def enqueue_def empty_def
  by (cases q) simp_all

end
```

We also present and experiment with Locales, adding infix notation, and precedence.

```
theory Experiments
  imports Main
```

```
begin

(*Locale definition of magma as a base theory definition*)
locale magma =
  fixes f :: "'a ⇒ 'a ⇒ 'a" (infixl "⊗" 70)

(*Locale definition of semigroup extending magma*)
locale semigroup = magma +
  assumes assoc: "(a ⊗ b) ⊗ c = a ⊗ (b ⊗ c)"

(*Locale definition of monoid extending semigroup*)
locale monoid = semigroup +
  fixes z :: "'a" ("e")
  assumes left_neutral [simp]: "e ⊗ a = a"
  assumes right_neutral [simp]: "a ⊗ e = a"

(*Locale definition of group extending monoid*)
locale group = monoid +
  fixes inv :: "'a ⇒ 'a" ("(_^-^1)" [1000] 999)
  assumes left_inverse [simp]: "inv a ⊗ a = e"
  assumes right_inverse [simp]: "a ⊗ inv a = e"

(*Sledgehammered proof of inverse_identity *)
lemma (in group) inv_identity [simp]: "inv e = e"
  by (metis Experiments.group.right_inverse
      Experiments.monoid.left_neutral group_axioms
      monoid_axioms)

end
```

I experimented with proof search, forcing Isabelle to find proof terms sufficient to establish my statement; this was done via the `sledgehammer` command.

# References


[1] Thomas Hales, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Le Truong Hoang, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Tat Thang Nguyen, Quang Truong Nguyen, Tobias Nipkow, Steven Obua, Joseph Pleso, Jason Rute, Alexey Solovyev, Thi Hoai An Ta, Nam Trung Tran, Thi Diep Trieu, Josef Urban, Ky Vu, and Roland Zumkeller. A formal proof of the Kepler conjecture. *Forum of Mathematics, Pi*, 5:e2, 2017.

[2] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220. ACM, 2009.

[3] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.

[4] Bhargav Shivkumar. Isabelle theorem prover tutorial. `https://www.youtube.com/watch?v=1nEpUoVopT0&list=PL88g1zsvCrjd5cbOLsN6NLn1yX7azcTS6`, 2020. Accessed: 2026.